

Berkeley DB

Porting Berkeley DB

Version 5.3.33

Copyright (c) 1990, 2013 Oracle and/or its affiliates. All rights reserved.

Preface

Table of Contents

[Conventions Used in this Book](#)

[Audience](#)

[For More Information](#)

[Contact Us](#)

The Berkeley DB family of open source, embeddable databases provides developers with fast, reliable persistence with zero administration. Often deployed as "edge" databases, the Berkeley DB family provides very high performance, reliability, scalability, and availability for application use cases that do not require SQL.

As an open source database, Berkeley DB works on many different platforms, from Wind River's Tornado system, to VMS, to Windows NT and Windows 95, and most existing UNIX platforms. It runs on 32 and 64-bit machines, little or big-endian.

Berkeley DB Porting Guide provides the information you need to port Berkeley DB 11g Release 2 (library version 11.2.5.3) to additional platforms.

Conventions Used in this Book

The following typographical conventions are used within in this manual:

Structure names are represented in `monospaced font`, as are `method names`. For example: "`DB->open()` is a method on a `DB` handle."

Variable or non-literal text is presented in *italics*. For example: "Go to your *DB_INSTALL* directory."

Program examples are displayed in a `monospaced font` on a shaded background. For example:

```
/* File: gettingstarted_common.h */
typedef struct stock_dbs {
    DB *inventory_dbp; /* Database containing inventory information */
    DB *vendor_dbp;    /* Database containing vendor information */

    char *db_home_dir; /* Directory containing the database files */
    char *inventory_db_name; /* Name of the inventory database */
}
```

```
char *vendor_db_name;    /* Name of the vendor database */
} STOCK_DBS;
```

Note

Finally, notes of interest are represented using a note block such as this.

Audience

This guide is intended for programmers porting Berkeley DB to a new platform. It assumes that these programmers possess:

- Familiarity with standard ANSI C and POSIX C 1003.1 and 1003.2 library and system calls.
- Working knowledge of the target platform as well as the development tools (for example, compilers, linkers, and debuggers) available on that platform.

For More Information

Contact Us

Beyond this manual, you may also find the following sources of information useful when building a DB application:

- [Getting Started with Berkeley DB for C](#)
- [Getting Started with Transaction Processing for C](#)
- [Berkeley DB Getting Started with Replicated Applications for C](#)
- [Berkeley DB Installation and Build Guide](#)
- [Berkeley DB Programmer's Reference Guide](#)
- [Berkeley DB Getting Started with the SQL APIs](#)
- [Berkeley DB C API Reference Guide](#)

To download the latest Berkeley DB documentation along with white papers and other collateral, visit <http://www.oracle.com/technetwork/indexes/documentation/index.html>.

For the latest version of the Oracle Berkeley DB downloads, visit <http://www.oracle.com/technetwork/database/berkeleydb/downloads/index.html>.

Contact Us

You can post your comments and questions at the Oracle Technology (OTN) forum for Oracle Berkeley DB at: <http://forums.oracle.com/forums/forum.jspa?forumID=271>, or for Oracle Berkeley DB High Availability at: <http://forums.oracle.com/forums/forum.jspa?forumID=272>.

For sales or support information, email to: berkeleydb-info_us@oracle.com You can subscribe to a low-volume email announcement list for the Berkeley DB product family by sending email to: bdb-join@oss.oracle.com

Chapter 1. Introduction to Porting Berkeley DB

Table of Contents

[Types of Berkeley DB ports](#)

[When Oracle Has Agreed to Support Berkeley DB on the New Platform](#)

[When Oracle has Not Agreed to Support Berkeley DB on the New Platform](#)

[Berkeley DB Porting Process](#)

Berkeley DB is an open source database product that supports a variety of platforms. When there is a need to run Berkeley DB on a platform that is currently not supported, DB is distributed in source code form that you can use as base source to port Berkeley DB to that platform.

Berkeley DB is designed to be as portable as possible, and has been ported to a wide variety of systems, from Wind River's Tornado system, to VMS, to Windows/NT and Windows/95, and most existing UNIX platforms. It runs on 16-bit, 32-bit, and 64-bit machines, little or big-endian. The difficulty of a port depends on how much of the ANSI C and POSIX 1003.1 standards the new architecture offers.

Before you begin actually porting Berkeley DB, you need an understanding of the:

- [Types of Berkeley DB ports](#)
- [Berkeley DB Porting Process](#)

Types of Berkeley DB ports

[When Oracle Has Agreed to Support Berkeley DB on the New Platform](#)

[When Oracle has Not Agreed to Support Berkeley DB on the New Platform](#)

There are several types of Berkeley DB ports:

- Ports developed and supported by Oracle
- Ports developed by a customer or a partner, but which Oracle has agreed to support.
- Ports developed, maintained, and supported by a customer or partner.

For a port developed by a customer or a partner, the general steps for porting Berkeley DB to a new platform are the same whether or not Oracle has agreed to support Berkeley DB on the new platform. For example, after you complete the port you send it to Berkeley DB as described in [Integrating Changes into the Berkeley DB Source Code](#). However, there are some differences.

When Oracle Has Agreed to Support Berkeley DB on the New Platform

When porting Berkeley DB to a platform that Oracle has agreed to support, you need to have Berkeley DB engineering review your port at various points. These review points are discussed more fully in [Integrating Changes into the Berkeley DB Source Code](#), [Modifying the Tests](#), and [Reviewing the Results of the Tests](#).

It is up to you to submit the results of the tests (test_micro, test_mutex, and, if possible, the entire tcl test suit) for review by Oracle Berkeley DB engineering in order for Oracle to consider providing support for Berkeley DB on a new platform.

You must also assign copyrights for your changes to any part of Berkeley DB to "Oracle Corporation" and attest to the fact that you are not infringing on any software patents for the changes to be included in the general Berkeley DB distribution.

Once the port is certified, Oracle provides support for Berkeley DB on the new platform in the same manner that it does for Berkeley DB running on other established platforms.

When Oracle has Not Agreed to Support Berkeley DB on the New Platform

When Oracle has *not* agreed to support Berkeley DB on the new platform, the customer or partner assume the responsibility of front-line support. When it is determined that there is a problem in the code that was not modified by the customer or partner, then Berkeley DB engineering provides support to the customer or vendor who implemented the port. However, in these cases, Oracle needs access to the platform and hardware for diagnosing, debugging, and testing.

Berkeley DB Porting Process

As with any porting project, porting Berkeley DB to a new platform consists of the following process:

1. Determine the modifications that you need to make in the base code and create a working executable of Berkeley DB on the target platform as described in [Creating a New Berkeley DB Binary](#).
2. Perform final test and quality assurance functions on the target platform as described in [Testing and Certifying the Port](#).

Chapter 2. Creating a New Berkeley DB Binary

Table of Contents

[Creating a Base Build of Berkeley DB](#)

[Determining the Scope of the Modifications](#)

[Do Changes Need to be Made to the Operating System Functionality?](#)

[Are Some Standard Functions Missing on the Target Platform?](#)

[How Will the Port Handle Shared Memory?](#)

[What Type of Mutexes Will the Port Use?](#)

[Do Any Other Changes Need to be Made?](#)

[Building on the Target Platform](#)

[Source Code Layout](#)

Creating a new Berkeley DB executable on the target platform, involves:

1. [Creating a Base Build of Berkeley DB](#)
2. [Determining the Scope of the Modifications](#)
3. [Building on the Target Platform](#)

Creating a Base Build of Berkeley DB

The simplest way to begin a port is to attempt to configure and build Berkeley DB on a UNIX or UNIX-like system. This gives you a list of the files that you needed to build Berkeley DB as well as the configuration files you can use as a starting point for building on your target port.

To create a base build of Berkeley DB, following the instructions in the *Berkeley DB Programmer's Reference Guide*:

1. Download a Berkeley DB distribution from <http://www.oracle.com/technetwork/database/berkeleydb/downloads/index.html>.
2. Build Berkeley DB.

Berkeley DB uses the GNU autoconf tools for configuration on almost all of the platforms it supports. Specifically, the include file `db_config.h` configures the Berkeley DB build. The simplest way to begin a port is to configure and build Berkeley DB on a UNIX or UNIX-like system, and then take the `Makefile` and `db_config.h` file created by that configuration, and modify it by hand to reflect the needs of the new architecture. Unless you are already familiar with the GNU autoconf toolset, we do not recommend you take the time to integrate your changes back into the Berkeley DB autoconfiguration framework. Instead, send us context diffs of your changes and any new source files you created, and we can integrate the changes into our source tree.

Determining the Scope of the Modifications

[Do Changes Need to be Made to the Operating System Functionality?](#)

[Are Some Standard Functions Missing on the Target Platform?](#)

[How Will the Port Handle Shared Memory?](#)

[What Type of Mutexes Will the Port Use?](#)

[Do Any Other Changes Need to be Made?](#)

Once you have a good build of Berkeley DB on a UNIX or UNIX-like system, look over the code to determine what type of code changes you need to make so that you can successfully build Berkeley DB on your target system. This process involves determining:

- [Do Changes Need to be Made to the Operating System Functionality?](#)
- [Are Some Standard Functions Missing on the Target Platform?](#)
- [How Will the Port Handle Shared Memory?](#)

- [What Type of Mutexes Will the Port Use?](#)
- [Do Any Other Changes Need to be Made?](#)

Do Changes Need to be Made to the Operating System Functionality?

Berkeley DB uses about forty operating system primitives. The Berkeley DB distribution contains files which are wrappers around these operating system primitives that act as an abstraction layer to separate the main Berkeley DB code from operating system and architecture-specific components. You *must* port these files (or versions of these files) whenever you port Berkeley DB to a new platform.

Within a Berkeley DB distribution, typically, there is only a single version of these files for all platforms that Berkeley DB supports. Those versions of the files live in the `os` directory of the distribution and follow the ANSI C and POSIX 1003.1 standards. Within each file, there is usually one, but sometimes several functions (for example, the `os_alloc.c` file contains functions such as `malloc`, `realloc`, `strdup`, `free`). The following table describes the files in the `os` directory of the Berkeley DB distribution along with the POSIX functions that must be ported.

POSIX Functions	Internal Function Name	Source File
<code>abort()</code> is required if diagnostic build is used or if <code>snprintf</code> is not provided by the platform	<code>__os_abort()</code>	<code>os_abort.c</code>
<code>freeaddrinfo()</code> <code>getaddrinfo()</code> , <code>htonl()</code> , <code>htons()</code> , <code>inet_addr()</code> , and <code>gethostbyname()</code> are required for Replication Manager	<code>__os_getaddrinfo()</code> , <code>__os_freeaddrinfo()</code>	<code>os_addrinfo.c</code>
<code>malloc()</code> , <code>realloc()</code> , <code>strdup()</code> , <code>free()</code> , <code>memcpy()</code> , <code>memset()</code> , <code>strlen()</code>	<code>__os_umalloc()</code> , <code>__os_urealloc()</code> , <code>__os_ufree()</code> , <code>__os_strdup()</code> , <code>__os_calloc()</code> , <code>__os_malloc()</code> , <code>__os_realloc()</code> , <code>__os_free()</code> , <code>__os_guard()</code> , <code>__ua_memcpy()</code>	<code>os_alloc.c</code>
<code>clock_gettime()</code> , <code>time()</code> , <code>gettimeofday()</code>	<code>__os_gettime()</code>	<code>os_clock.c</code>
<code>sysconf()</code>	<code>__os_cpu_count()</code>	<code>os_cpu.c</code>

POSIX Functions	Internal Function Name	Source File
ctime(), ctime_r()	__os_ctime()	os_ctime.c
opendir(), closedir(), readdir(), stat()	__os_dirlist(), __os_dirfree()	os_dir.c
strncpy()	__os_get_errno_ret_zero(), __os_get_errno(), __os_get_syserr(), __os_set_errno(), __os_strerror(), __os_posix_err()	os_errno.c
fcntl() is required for DB_REGISTER	__os_fdlock()	os_flock.c
fsync(), fdatasync()	__vx_fsync(), __os_fsync()	os_fsync.c
getenv() and strcpy() are required when environment variables are used to configure the database	__os_getenv()	os_getenv.c
close(), open()	__os_openhandle(), __os_closehandle()	os_handle.c
getpid() pthread_self() is required for replication and failchk functionality	__os_id()	os_pid.c
shmget(), shmdt(), shmctl(), and shmat() are required when environment uses share memory for regions munmap() is required when environment uses memory mapped files for regions or read-only databases munlock() is required when environment is configured with DB_LOCKDOWN	__os_attach(), __os_detach(), __os_mapfile(), __os_unmapfile(), __os_map(), __shm_mode(), __no_system_mem()	os_map.c
mkdir(), chmod()	__os_mkdir()	os_mkdir.c

POSIX Functions	Internal Function Name	Source File
fchmod() directio() is required when explicitly enabling DIRECTIO_ON	__os_open()	os_open.c
rename()	__os_rename()	os_rename.c
getuid() is required when environment variables are used to configure the database	__os_isroot()	os_root.c
read(), write(), pread(), pwrite()	__os_io(), __os_read(), __os_write(), __os_physwrite()	os_rw.c
lseek()	__os_seek()	os_seek.c
stat(), fstat()	__os_exists(), __os_ioinfo()	os_stat.c
ftruncate() is required when using truncate	__os_truncate()	os_truncate.c
unlink()	__os_unlink()	os_unlink.c
yield(), sched_yield()	__os_yield(), __os_sleep()	os.yield.c

When the operating system primitives on the target platform are identical or close to the POSIX semantics that Berkeley DB requires, then no code changes or minimal code changes to the files in the `os` directory are required. If the operating system primitives are quite different, then some code changes may be required to bridge the gap between the requirements of Berkeley DB and what the operating system provides.

Where different code is required, you write an entirely different version of the file and place it in an `os_xxx` directory where `xxx` represents a platform name. There are `os_xxx` subdirectories in the Berkeley DB distribution for several established non-POSIX platforms. For example, there is a `os_vxworks` directory that contains VxWorks versions of some of the files in the `os` directory, and Windows versions of some files are in the `os_windows` directory. If your

target platform needs a different version of a file, you will need to write that file and place it in a new `os_XXX` directory that you create for your target platform.

Are Some Standard Functions Missing on the Target Platform?

In some cases, the target platform may not provide the few POSIX functions required by Berkeley DB or the functions provided by the target platform may not operate in a standard compliant way. Berkeley DB provides replacement functions in the `clib` directory of the Berkeley DB distribution.

You need to determine how your target platform handles these functions:

- When the target platform does *not* have a POSIX function required by Berkeley DB, no action is required on your part. When Berkeley DB cannot find one of these functions on the target platform, it automatically uses the replacement functions supplied in the `clib` directory of the Berkeley DB distribution. For example, if the target platform does not have the `atoi` or `strtol` functions, Berkeley DB uses `clib/atoi.c` and `clib/strtol.c`.
- When the target platform has a function required by Berkeley DB, but that function operates in a non-standard compliant way, you can code to the replacement functions supplied in the `clib` directory.

How Will the Port Handle Shared Memory?

In order to write multiprocess database applications (not multithreaded, but threads of control running in different address spaces), Berkeley DB must be able to name pieces of shared memory and access them from multiple processes.

On UNIX/POSIX systems, Berkeley DB uses `mmap` and `shmget` for that purpose, but any interface that provides access to named shared memory is sufficient. If you have a simple, flat address space, you should be able to use the code in `os_vxworks/os_map.c` as a starting point for the port.

If you are not intending to write multiprocess database applications, then this won't be necessary, as Berkeley DB can simply allocate memory from the heap if all threads of control will live in a single address space.

What Type of Mutexes Will the Port Use?

Berkeley DB requires some form of self-blocking mutual exclusion mutex. Blocking mutexes are preferred as they tend to be less CPU-expensive and less likely to cause thrashing. If blocking mutexes are not available, however, test-and-set will work as well. The code for mutexes is in two places in the system: the include file `dbinc/mutex_int.h`, and the distribution directory `mutex`.

Do Any Other Changes Need to be Made?

In most cases, you do not need to make any changes to the Berkeley DB source code that is not in the abstraction layer (that is, in the `os` directory) as that code is designed to be platform-independent code. However, in some situations, the compiler for the target platform is non-standard and may raise errors when compiling some aspects of the Berkeley DB code (for example, additional casting may be required, or a certain type may cause a problem). In these cases, you will need to modify the generic Berkeley DB code in order to have error-free compilation.

Building on the Target Platform

Once you have an idea of the scope of the modifications, use the files generated on the UNIX system to help you create any compiler or linker tool chain files that you need to build on the target platform. At this point, start building the Berkeley DB on the target platform making the changes to the code that you identified earlier in the process. Once you have identified the modifications that you need to make, change the code accordingly.

Source Code Layout

The following table describes the directories in the Berkeley DB distribution.

Directory	Description
LICENSE	Berkeley DB License
build_android	Android build directory
build_unix	UNIX build directory
build_vxworks	VxWorks build directory
build_wince	Windows CE build directory
build_windows	Windows build directory
dist	Scripts used to auto-generate code for Berkeley DB distribution and administration
docs	Documentation
examples	Example programs for various language APIs
lang	Implementation of various APIs that work with the Berkeley DB library
src	Implementation of the Berkeley DB library
test	Test suites
util	Implementation of utilities that can be used with Berkeley DB

Chapter 3. Testing and Certifying the Port

Table of Contents

[Types of Tests for Berkeley DB](#)

[Modifying the Tests](#)

[Running the Tests](#)

[Reviewing the Results of the Tests](#)

[Integrating Changes into the Berkeley DB Source Code](#)

[Certifying a Port of Berkeley DB](#)

There are several different types of tests available for validating your port of Berkeley DB as discussed in [Types of Tests for Berkeley DB](#). Testing your port involves:

- [Modifying the Tests](#)
- [Running the Tests](#)
- [Reviewing the Results of the Tests](#)
- [Integrating Changes into the Berkeley DB Source Code](#)
- [Certifying a Port of Berkeley DB](#)

Types of Tests for Berkeley DB

There are two types of tests available for testing your port of Berkeley DB:

- The C Tests for Berkeley DB

There are two types of C tests for Berkeley DB. Each of these is in its own directory:

- `test_mutex` contains files that test the use of mutexes in Berkeley DB.
- `test_micro` contains the C tests that exercise the most common code paths, but it is not intended to be an exhaustive Test Suite. Additionally, it tests the different versions of Berkeley DB (including the new port) against each other. The `test_micro` tests can either be run in a shell or as simple C tests.
- The Berkeley DB Test Suite

The `test` directory contains the Berkeley DB Test Suite that tests all of the code in Berkeley DB. Using the Test Suite involves using Tool Command Language (Tcl) version 8.5 or later. Running the standard version of the Test Suite executes tests the major functionality of Berkeley DB. A more exhaustive version of the Test Suite runs all the tests several more times, testing encryption, replication, and different page sizes.

Note

Contact the Oracle Berkeley DB engineering team for a platform compatibility test suite.

Modifying the Tests

There should be no need to make modifications to the tests. However, in a few situations, the test hardware may have some constraints (for example, small amount of memory) which may cause certain tests (which expect more memory) to fail. In these rare situations, you may need to modify the tests to work within the constraints of the platform.

When Oracle has agreed to support Berkeley DB on the new platform, submit any proposed test changes for review and approval by Oracle Engineering before running the tests.

Running the Tests

You test your new port of Berkeley DB by running the tests in the following order:

1. Run the C tests in the following order:
 1. Tests for mutexes located in the `test_mutex` directory. To run the tests, follow the instructions in the `test_mutex/readme` file.
 2. Tests for the common code paths located in the `test_micro` directory. To run the tests in a shell script, follow the instructions in the `test_micro/readme` file. To run the tests as simple C tests, follow the instructions in the `test_micro/readme_embedded` file.
2. If the target platform supports the use of Tcl (version 8.5 or later), run the Test Suite. How you run the Test Suite varies depending on the target platform:
 - If the target platform supports a UNIX-like version of Tcl, then set up Tcl and build the Test Suite as described in "Running the Test Suite under UNIX" in *Berkeley DB Installation and Build Guide* at http://download.oracle.com/docs/cd/E17076_02/html/installation/build_unix_test.html and, then, run the test suite.
 - If the target platform supports a Windows-like version of Tcl, then setup Tcl, and build and run the Test Suite as described in "Running the Test Suite under

Windows" in *Berkeley DB Programmer's Reference Guide* at http://download.oracle.com/docs/cd/E17076_02/html/installation/build_win_test.html

Reviewing the Results of the Tests

It is up to you to submit the results of the tests (`test_micro`, `test_mutex`, and, if possible, the entire tcl test suite) for review by Oracle Berkeley DB engineering in order for Oracle to consider providing support for Berkeley DB on a new platform.

When Oracle has *not* agreed to support Berkeley DB on the new platform, you are responsible for ensuring that the tests run successfully.

Integrating Changes into the Berkeley DB Source Code

Once you have completed your port, send all code changes that you made as "diff" files to Berkeley DB development for review.

Exactly how you integrate changes into the Berkeley DB source code varies depending on whether or not Oracle has agreed to support Berkeley DB on the new platform:

At this point, when Oracle has agreed to support Berkeley DB on the new platform:

1. Berkeley DB development reviews the changes, makes the changes (with modifications if necessary) in the source code, creates another snapshot which includes the new platform-specific changes, and sends you the new snapshot.
2. On receipt of the new snapshot, recompile this new snapshot. If the new snapshot compiles correctly without any changes, test the port again.

Certifying a Port of Berkeley DB

When the target platform supports using Tcl, the port is considered certified after a successful standard run (`run_std`) of the Test Suite .

When the target platform does not support using Tcl, a port is considered certified if you see successful message reports for each of the tests in `test_micro` and `test_mutex`.

Additionally, the configuration and compilation of Berkeley DB must be complete without errors or warnings of any kind. You must provide specific information about the hardware, software, and compiler used during the porting process, especially versions of all third party tools used. If you have other diagnostic tools available, such as memory allocation checking tools, please conduct tests using them as well and provide Oracle engineering with the results. Finally, do not constrain your testing to one configuration. There are many different ways to

configure Berkeley DB (that is many options to the "configure" script), please configure, built, and test Berkeley DB on the target platform with as many combinations of these flags as possible to ensure that nothing is missed.